

Seminare ASC - Rezolvări

Matei-Iulian Cocu

`matei-iulian.cocu@s.unibuc.ro`

Ștefan Chiper

`stefan.chiper@s.unibuc.ro`

Cuprins

Seminar 0x00	2
Seminar 0x01	5
Seminar 0x02	9
Seminar 0x03	10
Seminar 0x04	11
Seminar 0x05	11

Seminar 0x00

1. Completați următorul tabel cu reprezentările lipsă ale unor numere naturale:

Baza: $B = 2$	$B = 4$	$B = 8$	$B = 10$	$B = 16$
1011 1110 1110 1111	2332 3233	137 357	48879	0xBEEF
0000 0000 0010 1010	222	52	42	0x002A
1001 1100 1111 0011	2130 3303	116 363	40179	0x9CF3
0000 0010 1011 1101	22331	1275	701	0x02BD
0000 0001 1111 1111	13333	777	511	0x01FF
1101 1110 1010 1111	3132 2233	157 257	57007	0xDEAF
1100 1010 0101 0011	3022 1103	145 123	51795	0xCA53
0000 0000 1110 0100	3210	344	228	0x00E4
0000 1110 1001 1101	322131	7235	3741	0x0E9D
0000 0111 1110 0100	133210	3744	2020	0x07E4
0001 0001 0001 0001	101 0101	10 421	4,369	0x1111
1111 1111 1111 1111	3333 3333	177 777	65535	0xFFFF
1111 1111 0000 0000	3333 0000	177 400	65280	0xFF00
0000 0000 1111 1111	3333	377	255	0x00FF

2. Completați următorul tabel cu reprezentările lipsă ale unor numere întregi (16 biți):

Baza: $B = 2$	$B = 4$	$B = 8$	$B = 10$	$B = 16$
1111 1110 1110 1101	3332 3231	177 355	-275	0xFEED
1111 1111 1101 0110	3333 0332	377 326	-42	0xFFD6
0101 1100 1111 0011	1130 3303	057 363	23795	0x5CF3
0000 0010 1011 1101	22331	2535	701	0x02BD
0000 0011 1111 1111	3333	777	511	0x01FF
1010 0001 1111 0001	2023 3011	243 361	-24111	0xA1F1
1100 1010 0101 0011	3102 1123	145 123	-13645	0xCA53
0000 0000 1110 0100	3210	344	228	0x00E4
0000 1110 0000 0111	320 013	7007	3591	0x0E07
1111 1010 1011 0111	3322 2313	3667	-1337	0xFAB7
1111 0001 0001 0001	3302 2211	170 421	-3855	0xF111
1111 1111 1111 1111	3333 3333	177 777	-1	0xFFFF
1111 1111 0000 0000	3333 0000	177 400	-256	0xFF00
1111 0000 1111 0000	3300 3300	170 360	-3856	0xF0F0

3. Realizați următoarele operații aritmetice cu numere pe biți reprezentate în complement față de doi (verificați-vă calculele folosind sistemul zecimal):

0101 1100 1111 0011 1111 1111 0000 0000 overflow - 0101 1011 1111 0011	1111 1111 1111 1111 0000 0000 0000 0001 overflow - 0000 0000 0000 0000
0111 1111 1111 1111 0000 0000 0000 0001 1000 0000 0000 0000	0101 1100 1111 0011 0111 0000 1111 0000 1100 1101 1110 0011
1111 1111 1111 1111 0000 0000 0000 0000 1111 1111 1111 1111	1000 0000 0000 0000 0000 0000 0000 0001 1000 0000 0000 0001

În următoarele cazuri, numărul de biți scriși reprezintă dimensiunea totală alocată pentru reprezentarea numerelor (14 și 8 biți în stânga, 12 și 8 biți în dreapta):

$$\begin{array}{r|l} 11\ 1000\ 0010\ 0101 & \\ \hline 1011\ 1100 & + \\ \hline 11\ 1000\ 1110\ 0001 & \end{array}$$

$$\begin{array}{r|l} 1100\ 1001\ 0010 & \\ \hline 0100\ 1010 & + \\ \hline 1100\ 1101\ 1100 & \end{array}$$

4. Realizați următoarele operații logice pe biți:

$$\begin{array}{r|l} 0101\ 1100\ 1111\ 0011 & \\ \hline 0101\ 1100\ 1111\ 0011 & AND \\ \hline 0101\ 1100\ 1111\ 0011 & \end{array}$$

$$\begin{array}{r|l} 0111\ 1111\ 1111\ 1111 & \\ \hline 0000\ 0000\ 0000\ 0001 & AND \\ \hline 0000\ 0000\ 0000\ 0001 & \end{array}$$

$$\begin{array}{r|l} 1101\ 1001\ 0110\ 0001 & \\ \hline 1111\ 1111\ 0000\ 0000 & AND \\ \hline 1101\ 1001\ 0000\ 0000 & \end{array}$$

$$\begin{array}{r|l} 0000\ 0000\ 1111\ 1111 & \\ \hline 0000\ 0001\ 0000\ 0000 & AND \\ \hline 0000\ 0000\ 0000\ 0000 & \end{array}$$

$$\begin{array}{r|l} 1101\ 1100\ 1111\ 0011 & \\ \hline 1101\ 1100\ 1111\ 0011 & XOR \\ \hline 0000\ 0000\ 0000\ 0000 & \end{array}$$

$$\begin{array}{r|l} 0101\ 1100\ 1111\ 0011 & \\ \hline 0111\ 0000\ 1111\ 0000 & OR \\ \hline 0111\ 1100\ 1111\ 0011 & \end{array}$$

$$\begin{array}{r|l} 0101\ 1100\ 1111\ 0011 & \\ \hline 0000\ 0000\ 1111\ 1111 & OR \\ \hline 0101\ 1100\ 1111\ 1111 & \end{array}$$

$$\begin{array}{r|l} 1100\ 0110\ 1001\ 1110 & \\ \hline 1001\ 1111\ 0110\ 1100 & XOR \\ \hline 1100\ 0110\ 1001\ 1110 & XOR \\ \hline 1001\ 1111\ 0110\ 1100 & \end{array}$$

5. Folosind cunoștințele de curs, răspundeți la următoarele întrebări:

- care este cel mai mare număr zecimal (natural) care se poate reprezenta pe N biți? **R:** $2^N - 1$
- care este cel mai mare număr zecimal (întreg) care se poate reprezenta (în complement față de doi) pe N biți? dar cel mai mic? **R:** $2^{N-1} - 1$ (mare); -2^{N-1} (mic)
- fie x un număr natural, de câți biți este nevoie pentru a-l reprezenta în binar? **R:** $\lceil \log_2(x) \rceil + 1$
- dacă un număr x este reprezentat cu k cifre în sistemul hexazecimal, de câți biți avem nevoie pentru a-l reprezenta în binar? **R:** $4k$
- dacă un număr x este reprezentat cu k biți în sistemul binar, de câte cifre avem nevoie pentru a reprezenta x în sistemul hexazecimal? **R:** $\frac{k}{4}$
- dacă un număr x este reprezentat cu k cifre în sistemul zecimal, de câți biți avem nevoie pentru a-l reprezenta în binar? **R:** $\lceil k \cdot \log_2(10) \rceil$

6. Reprezentarea binară a unui număr poate fi extinsă la dreapta, cu numere sub-unitare în felul următor (binary fixed-point):

...	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	...
-----	-------	-------	-------	-------	-------	-------	-------	-------	----------	----------	----------	----------	----------	----------	----------	-----

- calculați valorile în sistemul zecimal pentru următoarele numere reprezentate în formatul de mai sus (punctul stă între 2^0 și 2^{-1}):
 - 101.101 **R:** 5.625
 - 111.001 **R:** 7.125
 - 1110.00111 **R:** 14.21875
 - 1010.0000011 **R:** 10.0234375
 - 1111.0010011 **R:** 15.148438
- calculați valorile în sistemul binar fixed-point pentru următoarele numere reprezentate în formatul zecimal:
 - 3.75 **R:** 11.11
 - 12.3125 **R:** 1100.0101
 - 3.078125 **R:** 11.000101
 - 17.671875 **R:** 10001.101011
 - $\frac{2}{3} \approx 0.6667$ **R:** 0.1010101010... (periodic)
- când au numere reprezentate în acest sistem o reprezentare finită (fără repetiția zecimalelor)? **R:** Atunci când partea fracționară poate fi exprimată ca o sumă finită de puteri negative ale lui 2, adică atunci când numărul fracționar este un multiplu finit al unei puteri negative a lui 2. Altfel, numerele care nu pot fi exprimate astfel vor avea o reprezentare periodică în binar.
- cum arată un număr negativ reprezentat într-un astfel de sistem de reprezentare? Reprezentați $\frac{5}{8}$ și $-\frac{5}{8}$ în sistemul binary fixed-point (folosiți un sigur bit pentru reprezentarea părții întregi, gândiți-vă și cum folosiți un bit de semn și la reprezentarea în complement față de doi). **R:** Vom alocă 8 biți (1 byte) pentru reprezentarea unui număr întreg. Similar cu reprezentarea în complement față de 2 a numerelor întregi, aceeași idee se aplică și pentru numerele din sistemul fixed-point: se calculează complementul față de 2 a numărului non-negativ și se adaugă 1 la acesta (un LSB în reprezentarea respectivă). Astfel, $\frac{5}{8} = 0.625 \rightarrow 0b0000.1010 \Rightarrow -\frac{5}{8} = -0.625 \rightarrow 0b1111.0110 = 0b1111.0101 + 0b0000.0001$.

7. Demonstrați cum reprezentarea în complement față de doi a numărului $-x$ poate fi obținută prin inversarea tuturor biților lui x și adunarea lui 1 la rezultat.

R: Fie x un număr întreg pozitiv reprezentat pe N biți. Reprezentarea sa binară este dată de suma:

$$x = \sum_{i=0}^{N-1} b_i \cdot 2^i, \quad b_i \in \{0, 1\}$$

Atunci, reprezentarea în complement față de doi a numărului $-x$ este dată de:

$$-x = 2^N - x = 2^N - \sum_{i=0}^{N-1} b_i \cdot 2^i = \sum_{i=0}^{N-1} (1 - b_i) \cdot 2^i + 1$$

unde $\sum_{i=0}^{N-1} (1 - b_i) \cdot 2^i$ reprezintă inversarea tuturor biților lui x , iar adunarea lui 1 finalizează procesul de obținere a reprezentării în complement față de doi.

8. Demonstrați că metoda asocierii a p termeni consecutivi este soluția corectă pentru problema transformării unui număr din baza B în baza B^p , $p > 1, p \in \mathbb{N}$.

R: Fie un număr N reprezentat în baza B ca o secvență de cifre $d_k d_{k-1} \dots d_1 d_0$, unde fiecare cifră d_i satisface $0 \leq d_i < B$. Atunci, valoarea acestui număr în baza 10 este dată de:

$$N = \sum_{i=0}^k d_i \cdot B^i$$

Pentru a transforma acest număr în baza B^p , împărțim secvența de cifre în grupuri de câte p cifre, începând de la dreapta spre stânga. Fiecare grup de p cifre reprezintă o cifră în baza B^p . Astfel, fiecare grup G_j format din cifrele $d_{jp+p-1} d_{jp+p-2} \dots d_{jp}$ poate fi convertit în baza B^p prin calculul:

$$C_j = \sum_{i=0}^{p-1} d_{jp+i} \cdot B^i$$

unde C_j este cifra corespunzătoare în baza B^p . Prin această metodă, fiecare grup de p cifre din baza B este transformat într-o singură cifră în baza B^p , păstrând astfel valoarea numerică a numărului original. Aceasta demonstrează că metoda asocierii a p termeni consecutivi este corectă pentru transformarea unui număr din baza B în baza B^p .

9. Demonstrați că $\lfloor \log_2 x \rfloor = i_{max}$ unde x este un număr dat pe N biți iar $i_{max} = \max\{i | b_i = 1, \forall i = 0, \dots, N - 1\}$ unde b_i reprezintă al i -lea bit din reprezentarea binară a numărului x . De exemplu, dacă $x = 00101110$ (46 zecimal) atunci $\lfloor \log_2 x \rfloor = 5$.

R: Fie x un număr reprezentat pe N biți, astfel încât:

$$x = \sum_{i=0}^{N-1} b_i \cdot 2^i, \quad b_i \in \{0, 1\}$$

Fie $i_{max} = \max\{i \mid b_i = 1, \forall i = 0, \dots, N - 1\}$. Atunci, putem observa că: $2^{i_{max}} \leq x < 2^{i_{max}+1}$ Aplicând logaritmul în baza 2 pe toate părțile inegalității, obținem: $i_{max} \leq \log_2(x) < i_{max} + 1$ Din această relație rezultă că: $\lfloor \log_2(x) \rfloor = i_{max}$.

10. În 2014, videoclipul melodiei "Gangnam Style" se apropia de 2.147.483.647 vizualizări (adică $2^{31} - 1$). Cei de la Youtube au trebuit să facă niște modificări pentru a acomoda acest număr mare de vizualizări. Explicați ce s-a întâmplat: care este semnificația numărului de mai sus, care era riscul, și care e soluția pentru rezolvarea potențiale probleme.

R: Numărul de vizualizări era stocat într-o variabilă de tip întreg pe 32 de biți, folosind reprezentarea în complement față de doi. Astfel, valoarea maximă care putea fi stocată era $2^{31} - 1 = 2.147.483.647$. Când numărul de vizualizări a atins această valoare, orice incrementare ar fi cauzat un overflow, resetând contorul la o valoare negativă sau zero, ceea ce ar fi fost incorect. Soluția a fost trecerea la o reprezentare pe 64 de biți pentru stocarea numărului de vizualizări, permițând astfel un număr mult mai mare de vizualizări fără riscul de overflow. De ce nu unsigned? regulile interne Youtube, pe scurt.

11. În Anexa 1 aveți două implementări ale algoritmului de căutare binară; care este cea optimă?

<pre>1 int binarySearch1(int arr[], int start, int end, int x) 2 { 3 if (end >= start) 4 { 5 int mid = start + (end - start) / 2; 6 if (arr[mid] == x) 7 return mid; 8 if (arr[mid] > x) 9 return binarySearch1(arr, start, mid - 1, x); 10 return binarySearch1(arr, mid + 1, end, x); 11 } 12 return -1; 13 }</pre>	<pre>1 int binarySearch2(int arr[], int start, int end, int x) 2 { 3 if (end >= start) 4 { 5 int mid = (start + end) / 2; 6 if (arr[mid] == x) 7 return mid; 8 if (arr[mid] > x) 9 return binarySearch2(arr, start, mid - 1, x); 10 return binarySearch2(arr, mid + 1, end, x); 11 } 12 return -1; 13 }</pre>
--	--

Explicație: în esență, singura diferență este modalitatea de calcul a punctului de mijloc: în prima variantă, este calculată media diferenței dintre capete (start și end) și este însumată punctului de plecare (start), pe când în cea de a doua variantă este făcută cât se poate de direct media celor 2 capete. Problema apare pentru cea de a DOUA variantă dacă ambele valori ale capetelor sunt mari, având posibilitatea de a face overflow; primul 'approach' atenuează această posibilitate complet.

Seminar 0x01

1. Avem un pachet de cărți de joc (52 de cărți). Luăm cărțile pentru prima dată afară din pachet. Câtă informație avem în acest moment despre cărți? Amestecăm cărțile aleator (apropro, cum facem asta, algoritmic, eficient?). Câtă informație avem acum în pachetul de cărți? (folosiți și aproximarea lui Stirling pentru calcularea rezultatului) **R:**

- **Informația inițială:** Când pachetul este nou, ordinea sa este cunoscută (o singură stare posibilă). Incertitudinea (entropia) este $H_0 = \log_2(1) = 0$ biți.
- **Algoritm de amestecare:** Cel mai eficient algoritm standard este **Fisher-Yates (sau Knuth)**, care parcurge vectorul și interschimbă fiecare element i cu un element aleator $j \leq i$. Are complexitate $O(n)$.
- **Informația după amestecare:** Odată amestecat, pachetul poate fi în oricare din $N = 52!$ stări (permutări) posibile. Entropia este:

$H_1 = \log_2(52!)$

Folosind aproximarea lui Stirling ($\log_2(n!) \approx n \log_2(n) - n \log_2(e) + \frac{1}{2} \log_2(2\pi n)$):

$H_1 \approx 52 \cdot \log_2(52) - 52 \cdot \log_2(e) + \frac{1}{2} \log_2(104\pi)$

$H_1 \approx 52 \cdot (5.70) - 52 \cdot (1.44) + \frac{1}{2}(8.35) \approx 296.4 - 75.0 + 4.18 \approx \mathbf{225.58}$ biți

2. Se dă o urnă în care avem 5 bile roșii și 3 bile albastre. Ni se spune că cineva extrage o bilă din urnă și aceasta este albastră. Cerinte:
- (a) câtă informație primim în urma acestei observații? **R: Total 8 bile. $P(A) = 3/8$. Informația $I(A) = -\log_2(P(A)) = -\log_2(3/8) = \log_2(8) - \log_2(3) = 3 - 1.585 = \mathbf{1.415}$ biți.**
 - (b) care a fost entropia urnei înainte de extragere? care este entropia urnei după extragere? **R: Înainte (5R, 3A): $H_0 = -[(5/8) \log_2(5/8) + (3/8) \log_2(3/8)] \approx -[0.625(-0.678) + 0.375(-1.415)] \approx \mathbf{0.954}$ biți. După (5R, 2A): Total 7 bile. $H_1 = -[(5/7) \log_2(5/7) + (2/7) \log_2(2/7)] \approx -[0.714(-0.485) + 0.286(-1.807)] \approx \mathbf{0.863}$ biți.**
 - (c) continuați să calculați entropia presupunând că extragem pas cu pas toate bilele albastre; **R:**
 - H_0 (5R, 3A) $\approx \mathbf{0.954}$ biți
 - H_1 (5R, 2A) $\approx \mathbf{0.863}$ biți
 - H_2 (5R, 1A): $H_2 = -[(5/6) \log_2(5/6) + (1/6) \log_2(1/6)] \approx \mathbf{0.651}$ biți
 - H_3 (5R, 0A): $H_3 = -[(1) \log_2(1) + 0] = \mathbf{0}$ biți
 - (d) similar cu cerința anterioară pentru bilele roșii (începând cu urna inițială presupunem că extrageți rând pe rând fiecare bilă roșie și calculați entropia la fiecare pas); **R:**
 - H_0 (5R, 3A) $\approx \mathbf{0.954}$ biți
 - H'_1 (4R, 3A): $H'_1 = -[(4/7) \log_2(4/7) + (3/7) \log_2(3/7)] \approx \mathbf{0.985}$ biți
 - H'_2 (3R, 3A): $H'_2 = -[(3/6) \log_2(3/6) + (3/6) \log_2(3/6)] = \mathbf{1}$ bit (Entropie maximă)
 - H'_3 (2R, 3A): $H'_3 = -[(2/5) \log_2(2/5) + (3/5) \log_2(3/5)] \approx \mathbf{0.971}$ biți
 - H'_4 (1R, 3A): $H'_4 = -[(1/4) \log_2(1/4) + (3/4) \log_2(3/4)] \approx \mathbf{0.811}$ biți
 - H'_5 (0R, 3A): $H'_5 = -[0 + (1) \log_2(1)] = \mathbf{0}$ biți
3. Se dau 12 bile. 11 dintre ele au aceeași greutate, iar una este mai ușoară sau mai grea decât toate celelalte. Aveți la dispoziție o balanță. Folosind un număr minim de cântăriri, găsiți bila diferită.

R:

- **Minim teoretic:** O cântărire are 3 rezultate (stânga grea, dreapta grea, echilibru). Avem $N = 12 \text{ bile} \times 2 \text{ stări} = 24$ stări posibile. Avem nevoie de k cântăriri astfel încât $3^k \geq 24$. Pentru $k = 2$, $3^2 = 9$ (insuficient). Pentru $k = 3$, $3^3 = 27$ (suficient). Numărul minim este **3 cântăriri**.
- **Algoritm:**
 1. Împărțim bilele în 3 grupe: $G1=1,2,3,4$, $G2=5,6,7,8$, $G3=9,10,11,12$.
 2. **Cântărirea 1: $G1$ vs $G2$.**
 3. **Caz A (Echilibru):** $G1 = G2$: Bila diferită e în $G3$. Știm că $G1$ e un set de bile normale.
 - **Cântărirea 2:** $9,10,11$ vs $1,2,3$ (normale).
 - **A.1 (Echilibru):** Bila e **12**. (O cântărire finală 12 vs 1 ne spune dacă e grea/ușoară).
 - **A.2 (Stânga grea):** Una din $9,10,11$ e grea. Cântărim 9 vs 10 . Dacă $9 \lessdot 10 \rightarrow \mathbf{9}$ e grea. Dacă $9 \dot{=} 10 \rightarrow \mathbf{10}$ e grea. Dacă $9 \gtrdot 10 \rightarrow \mathbf{11}$ e grea.
 - **A.3 (Stânga ușoară):** Una din $9,10,11$ e ușoară. Logică similară cu A.2.
 4. **Caz B (Stânga grea: $G1 > G2$):** Suspecți: $1,2,3,4$ (posibil grele) SAU $5,6,7,8$ (posibil ușoare). $G3$ e normală.
 - **Cântărirea 2:** $1,2,5$ vs $3,6,9$ (9 e normală).
 - **B.1 (Echilibru):** Suspecți rămași: 4 (grea), 7 (ușoară), 8 (ușoară). Cântărim 7 vs 8 . Dacă $7=8 \rightarrow \mathbf{4}$ e grea. Dacă $7 \lessdot 8 \rightarrow \mathbf{7}$ e ușoară. Dacă $7 \gtrdot 8 \rightarrow \mathbf{8}$ e ușoară.
 - **B.2 (Stânga grea):** Suspecți: 1 (grea), 2 (grea), 6 (ușoară). Cântărim 1 vs 2 . Dacă $1=2 \rightarrow \mathbf{6}$ e ușoară. Dacă $1 \lessdot 2 \rightarrow \mathbf{1}$ e grea. Dacă $1 \gtrdot 2 \rightarrow \mathbf{2}$ e grea.
 - **B.3 (Stânga ușoară):** Suspecți: 5 (ușoară), 3 (grea). Cântărim 5 vs 9 (normală). Dacă $5 \lessdot 9 \rightarrow \mathbf{5}$ e ușoară. Dacă $5=9 \rightarrow \mathbf{3}$ e grea.
 5. **Caz C (Stânga ușoară: $G1 < G2$):** Logică simetrică Cazului B.

4. Aveți în Curs 0x01 un slide despre A. Turing. Calculati entropia textului (nivel de caracter/simbol, fără grupări) din slide: prima dată cu și apoi fără diacritice. Comparati cele două valori calculate. **R: Nu am acces la documentul "Curs 0x01" pentru a analiza textul. Metodologia generală ar fi:**
- 1. Se numără frecvența absolută f_i a fiecărui caracter unic (simbol) din text.
 - 2. Se calculează probabilitatea $p_i = f_i/N_{total}$.
 - 3. Se calculează entropia $H = -\sum p_i \log_2(p_i)$.
 - 4. Se repetă pașii 1-3, mai întâi tratând literele cu diacritice (ex. 'ă') ca simboluri distincte de cele fără (ex. 'a'), și apoi tratându-le ca fiind identice (ex. 'ă' $\rightarrow$ 'a').
 - 5. Entropia calculată cu diacritice (alfabet mai mare și distribuție potențial diferită) va fi, cel mai probabil, diferită de cea fără diacritice.
5. Se dă un număr natural z pe N biți. Câtă informație am câștigat dacă:
- (a) ni se spune despre z că are exact două valori "1" în reprezentarea sa binară;
 - (b) ni se spune despre z că are exact $N/2$ valori "1" în reprezentarea sa binară;
 - (c) ni se spune despre z că are o secvență continuă de $N/4$ biți de "1" în reprezentarea sa binară (restul biților sunt "0");
 - (d) ni se spune despre z că are MSB setat la "1";
 - (e) ni se spune despre z că este impar;
 - (f) ni se spune despre z că este o putere a lui 2;
 - (g) ni se spune despre z că are primii $N/2$ biți din reprezentarea sa binară setați la "0";
 - (h) ni se spune despre z că este un număr prim (aici doar o estimare aproximativă este posibilă);
 - (i) ni se spune despre z că are în reprezentarea sa binară un număr par de biți setați la "1";
 - (j) ni se spune că $x = 42$.

R: Informația inițială (incertitudinea) este $H_0 = \log_2(2^N) = N$ biți. Informația câștigată este $I = H_0 - H_1$, unde $H_1 = \log_2(N_{stari_ramase})$ este entropia rămasă.

- (a) ...exact două valori "1": $N_{stari} = C(N, 2) = \frac{N(N-1)}{2}$. $I = N - \log_2(\frac{N(N-1)}{2})$.
- (b) ...exact $N/2$ valori "1": $N_{stari} = C(N, N/2)$. $I = N - \log_2(C(N, N/2))$.
- (c) ...secvență continuă de $N/4$ biți de "1" (restul "0"): Secvența poate începe la $N - N/4 + 1 = 3N/4 + 1$ poziții. $I = N - \log_2(3N/4 + 1)$.
- (d) ...MSB setat la "1": $N_{stari} = 1 \cdot 2^{N-1}$. $H_1 = \log_2(2^{N-1}) = N - 1$. $I = N - (N - 1) = 1$ bit.
- (e) ...este impar: LSB este '1'. $N_{stari} = 2^{N-1} \cdot 1$. $I = N - (N - 1) = 1$ bit.
- (f) ...este o putere a lui 2: Stările posibile sunt $\{2^0, 2^1, \dots, 2^{N-1}\}$. $N_{stari} = N$. $I = N - \log_2(N)$.
- (g) ...primii $N/2$ biți... "0": $N_{stari} = 1 \cdot 2^{N/2}$. $H_1 = N/2$. $I = N - N/2 = N/2$ biți.
- (h) ...număr prim (estimare): $\pi(x) \approx x/\ln(x)$. $N_{stari} = \pi(2^N) \approx \frac{2^N}{N \ln 2}$. $H_1 \approx \log_2(2^N/(N \ln 2)) = N - \log_2(N) - \log_2(\ln 2)$. $I = H_0 - H_1 \approx \log_2(N) + \log_2(\ln 2)$.
- (i) ...număr par de biți "1": Jumătate din numere (stările cu bit de paritate 0). $N_{stari} = 2^N/2 = 2^{N-1}$. $I = N - (N - 1) = 1$ bit.
- (j) ... $x = 42$: O singură stare. $N_{stari} = 1$. $H_1 = \log_2(1) = 0$. $I = N - 0 = N$ biți (toată incertitudinea a fost eliminată).

6. Se dă o variabilă aleatoare X care este distribuită conform următorului tabel: Cerinte:

- verificați dacă probabilitățile date sunt corecte (distribuția este consistentă);
- calculați entropia $H(X)$;
- construiți codul Huffman pentru X. Verificați eficiența acestui cod, comparați cu entropia;
- codificați mesajul ABBACED folosind codul Huffman găsit;
- asociați câte două evenimente/simboluri (vom numi această variabilă X^2). Cerinte:
 - câte evenimente/simboluri noi există acum? (care e dimensiunea variabilei X^2 ?);
 - calculați $H(X^2)$. Care este relația cu $H(X)$?
 - construiți codul Huffman pentru X^2 . Verificați eficiența acestui cod, comparați cu $H(X)$.

R: Simbolul 'E' are probabilitatea $1/2$, iar 'A', 'B', 'C', 'D' împart restul de $1/2$, deci au câte $1/8$ fiecare. **Distribuție:** $P(E)=0.5$, $P(A)=0.125$, $P(B)=0.125$, $P(C)=0.125$, $P(D)=0.125$.

- (a) ...verificați probabilitățile: $0.5 + 4 \times 0.125 = 0.5 + 0.5 = 1$. Distribuția este consistentă.
- (b) (b) ...calculați entropia $H(X)$: $H(X) = -[0.5 \log_2(0.5) + 4 \cdot (0.125 \log_2(0.125))] = -[0.5(-1) + 4 \cdot (0.125 \cdot -3)] = -[-0.5 - 1.5] = 2$ biți.
- (c) (c) ...construiți codul Huffman:
 - Combinăm (C: 0.125, D: 0.125) $\rightarrow$ CD: 0.25
 - Combinăm (A: 0.125, B: 0.125) $\rightarrow$ AB: 0.25
 - Combinăm (CD: 0.25, AB: 0.25) $\rightarrow$ ABCD: 0.5
 - Combinăm (E: 0.5, ABCD: 0.5) $\rightarrow$ 1.0**Cod:** E: 0, A: 110, B: 111, C: 100, D: 101. **Lungime medie (L):** $L = 0.5(1) + 4 \cdot (0.125 \cdot 3) = 0.5 + 1.5 = 2$ biți/simbol. **Comparație:** $L = H(X)$. Eficiența este 100%.
- (d) (d) ...codificați mesajul ABBACED: A(110) B(111) B(111) A(110) C(100) E(0) D(101) $\rightarrow$ 1101111111101000101
- (e) (e) ...asociați câte două evenimente/simboluri (X^2):
 - Nr. evenimente: $5 \times 5 = 25$ simboluri noi (AA, AB, ... EE).
 - $H(X^2)$ (presupunând independența): $H(X^2) = H(X) + H(X) = 2 \times 2 = 4$ biți.
 - Codul Huffman pentru X^2 ar avea o lungime medie $L_2 \geq H(X^2)$. Deoarece $L_1 = H(X)$, eficiența este deja perfectă, deci $L_2 = 4$ biți (sau 2 biți per simbol original).

7. Considerăm următorul mesaj bloc:

D_{00}	D_{01}	D_{02}	P_{0l}
D_{10}	D_{11}	D_{12}	P_{1l}
D_{20}	D_{21}	D_{22}	P_{2l}
P_{c0}	P_{c1}	P_{c2}	P_{cl}

Elementele D_{ij} sunt date (deci avem 9 biți) iar elementele P_{ij} sunt biți de paritate (deci avem 7 biți redundanți): pe fiecare linie, pe fiecare coloană și pe mesajul total. Biții de paritate sunt "1" dacă avem un număr par de "1" în linie/coloană/mesaj. Cerinte:

- să presupunem că bitul D_{01} se schimbă. Câți biți de paritate se schimbă?
- care este distanța Hamming minimă a codului de mai sus?
- câte erori putem detecta? Câte erori putem corecta?
- sunt următoarele mesaje valide sau corupte (dacă corupte, în ce fel)?

R: Definiția parității este neobișnuită (XNOR în loc de XOR). Presupunem că P_{cl} este paritatea întregului bloc de date $D_{00..22}$.

- (a) (a) ...bitul D_{01} se schimbă:
 - P_{0l} (paritatea liniei 0) se schimbă.
 - P_{c1} (paritatea coloanei 1) se schimbă.
 - P_{cl} (paritatea mesajului total) se schimbă.
 Se schimbă 3 biți de paritate (și bitul de date D_{01}).
- (b) (b) ...distanța Hamming minimă: Când D_{01} se schimbă, 4 biți în total se schimbă (D_{01} , P_{0l} , P_{c1} , P_{cl}). Distanța minimă a codului este $d=4$.
- (c) (c) ...câte erori putem detecta? Câte erori putem corecta?
 - Detectare: $d - 1 = 4 - 1 = 3$ erori.
 - Corectare: $\lfloor (d - 1)/2 \rfloor = \lfloor 3/2 \rfloor = 1$ eroare.
- (d) (d) ...mesaje valide sau corupte? (Tabelul este ilizibil. Folosind "1 pt par" pe un exemplu ipotetic: 011 1 / 101 1 / 010 0 / 101 1): **Mesaj Exemplu (011 1 / 101 1 / 010 0 / 101 1):**
 - L_0 (011 $\rightarrow$ 2 "1" e par): $P = 1$. Mesaj=1. OK.
 - L_1 (101 $\rightarrow$ 2 "1" e par): $P = 1$. Mesaj=1. OK.
 - L_2 (010 $\rightarrow$ 1 "1" e impar): $P = 0$. Mesaj=0. OK.
 - C_0 (010 $\rightarrow$ 1 "1" e impar): $P = 0$. Mesaj=1. **Eroare.**
 - C_1 (101 $\rightarrow$ 2 "1" e par): $P = 1$. Mesaj=0. **Eroare.**
 - C_2 (110 $\rightarrow$ 2 "1" e par): $P = 1$. Mesaj=1. OK.
 Mesajul este **corupt**. Erorile în P_{c0} și P_{c1} indică o posibilă eroare la intersecția lor, D_{11} și D_{10} , dar liniile par ok. E o eroare necorectabilă/multiplă.

8. Sistemul de codare ISBN-10 pentru cărți folosește 9 cifre plus o cifră de verificare pentru a identifica unic fiecare carte. Rolul cifrei de verificare este dublu: 1) detectează dacă o cifră din cele 10 este scrisă greșit și 2) detectează dacă două cifre consecutive au fost interschimbate. Aceste două erori sunt cele mai comune când codăm astfel de informații pentru cărți. Codare este următoarea: $x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, P$. (1) unde $x_1, \dots, x_9 \in \{0, \dots, 9\}$ sunt dați iar p (acesta ar fi x_{10}) este calculat astfel încât: $\sum_{i=1}^9 (11-i)x_i + p \equiv 0 \pmod{11}$. (2) Cerinte:
- calculați cifra de verificare p pentru codul ISBN-10: 0-306-40615p;
 - arătați că orice cod ISBN poate detecta orice eroare de o cifră;
 - arătați că orice cod ISBN poate detecta orice interschimbare a două cifre consecutive din cod;
 - care este distanța minimă pentru acest cod?
 - ce se întâmplă dacă schimbăm regula pentru cifra de verificare astfel încât $\sum_{i=1}^9 x_i + p \equiv 0 \pmod{11}$? Putem detecta/corecta erori la fel de bine?
 - verificați voi codul ISBN-13.

R: Formula: $\sum_{i=1}^9 (11-i)x_i + p \equiv 0 \pmod{11}$.

- (a) ...0-306-40615p:** $S = 10(0) + 9(3) + 8(0) + 7(6) + 6(4) + 5(0) + 4(6) + 3(1) + 2(5) = 0 + 27 + 0 + 42 + 24 + 0 + 24 + 3 + 10 = 130$. $130 = 11 \times 11 + 9 \implies 130 \equiv 9 \pmod{11}$. $9 + p \equiv 0 \pmod{11} \implies \mathbf{p = 2}$.
- (b) ...detecta orice eroare de o cifră:** O eroare la $x_j \rightarrow x'_j$ e nedetectată dacă $w_j(x'_j - x_j) \equiv 0 \pmod{11}$. Deoarece 11 e prim și $w_j \in [1, 10]$ și $(x'_j - x_j) \in [-9, 9] \setminus \{0\}$, niciun termen nu e 0 (mod 11). Produsul nu e 0. **Detectată.**
- (c) ...detecta orice interschimbare... consecutive:** Interschimbare $x_j \leftrightarrow x_{j+1}$. Nedetectată dacă $(w_j - w_{j+1})(x_j - x_{j+1}) \equiv 0 \pmod{11}$. $w_j = 11 - j$ și $w_{j+1} = 11 - (j+1) = 10 - j$. Deci $w_j - w_{j+1} = 1$. Ecuația devine $1 \cdot (x_j - x_{j+1}) \equiv 0 \pmod{11}$. Asta e adevărat doar dacă $x_j = x_{j+1}$, caz în care interschimbarea nu e o eroare. **Detectată.**
- (d) ...distanța minimă:** Codul detectează erori de 1 cifră ($d \geq 2$) și erori de 2 cifre (interschimbări) ($d > 2$). Distanța minimă este $\mathbf{d=3}$.
- (e) ...regula $\sum x_i + p \equiv 0 \pmod{11}$:** Toți $w_i = 1$. Eroarea de 1 cifră e detectată. Interschimbarea: $(w_j - w_{j+1})(x_j - x_{j+1}) = (1-1)(x_j - x_{j+1}) = 0$. Interschimbările nu ar fi detectate. Este un cod mai slab.
- (f) ...ISBN-13:** Folosește o sumă ponderată similară, dar (mod 10), cu ponderi alternante (1, 3, 1, 3, ...). Detectează erori de o cifră, dar nu toate interschimbările (doar cele între poziții cu ponderi diferite).

9. Considerăm utilizarea literelor în limba engleză. Conform Wikipedia cele 26 de litere ("a" la "z") apar cu frecvențele: 8.202.40Cerinte:
- câți biți am folosi pentru a coda alfabetul dat folosind șiruri binare cu dimensiune fixă?
 - calculați entropia acestui alfabet;
 - care ar fi entropia dacă toate literele ar avea aceeași frecvență? Care este relația cu entropia calculată pe alfabetul dat?
 - aveți posibilitatea de a înlătura din alfabetul de mai sus o literă. Eliminați litera astfel încât să minimizați/maximizați entropia noului alfabet (puteți să scrieți un program care să verifice, dar trebuie să înțelegeți cum actualizați toate probabilitățile atunci când eliminați o literă);
 - în Anexa 1 puteți verifica codul Morse. Considerăm că un punct ocupă un bit iar o linie ocupă doi biți de spațiu în memorie. Cerinte:
 - ce legătură observați între probabilitățile literelor și lungimea lor în cod Morse? decodați mesajul: -
 - mesajele codate în codul Morse se pot decoda unic? Puteți să decodați mesajul de mai sus dacă nu aveți spațiile după fiecare literă?
 - calculați lungimea medie pentru codarea alfabetului de mai sus în codul Morse, comparați cu entropia calculată anterior.

- (a) ...dimensiune fixă:** 26 litere. $2^4 = 16, 2^5 = 32$. Avem nevoie de **5 biți**.
- (b) ...entropia:** $H = -\sum p_i \log_2(p_i) = -[0.082 \log_2(0.082) + \dots + 0.0007 \log_2(0.0007)] \approx \mathbf{4.17}$ biți/caracter.
- (c) ...aceeași frecvență:** $H_{max} = \log_2(N) = \log_2(26) \approx \mathbf{4.70}$ biți/caracter. Entropia reală (4.17) e mai mică decât cea maximă (4.70), deoarece distribuția nu e uniformă.
- (d) ...înlătura... o literă:** (Probabilitățile trebuie re-normalizate: $p'_j = p_j / (1 - p_{\text{eliminat}})$).
 - Pentru a maximiza entropia: Eliminăm litera cu probabilitatea cea mai mare ('E', 13)
 - Pentru a minimiza entropia: Eliminăm o literă (de ex. 'T', 9.1)
- (e) ...Codul Morse:**
 - Legătura: Există o corelație puternică: literele frecvente ('E' 13)
 - Decodare '-.-': A.
 - Decodare unică? Da, dacă spațiile sunt prezente. Fără spații, nu. "... " poate fi 'S' sau 'I' urmat de 'E'. Mesajul "-.-" poate fi "AA" sau "C". Nu se pot decoda unic fără spații.
 - Lungime medie: (Presupunând 'linie'=2 biți, 'punct'=1 bit, și un spațiu '1 bit' între simboluri): $L_A = 1+2+1 = 4$ biți. $L_E = 1+1 = 2$ biți. $L = \sum p_i \cdot L_i$. Calculul complet ar da o valoare $L_{\text{Morse}} > H_{\text{entropie}}$ (deoarece codul Morse nu e optim binar).

10. Avem un alfabet cu două simboluri: simbolul "A" este codat pe un singur bit și are probabilitatea de apariție p și simbolul "B" este codat pe doi biți și are probabilitatea de apariție $1-p$ (bineînțeles). Cerinte:
- calculați lungimea medie a mesajelor cu alfabetul de mai sus;
 - definim rata entropiei ca fiind entropia supra lungimea medie a mesajelor. Găsiți p pentru care rata entropiei este minimizată/maximizată.

R: A: $L_A = 1, P(A) = p$. B: $L_B = 2, P(B) = 1 - p$.

- (a) ...lungimea medie:** $L = \sum P_i L_i = p(1) + (1-p)(2) = p + 2 - 2p = \mathbf{2 - p}$.
- (b) ...rata entropiei $R = H / L$:** $R(p) = \frac{-p \log_2 p - (1-p) \log_2 (1-p)}{2-p}$.
 - Minimizată: Rata R devine 0 când entropia H este 0. Asta se întâmplă la extreme: $\mathbf{p = 0}$ ($H=0, L=2, R=0$) și $\mathbf{p = 1}$ ($H=0, L=1, R=0$).
 - Maximizată: Se maximizează când $R'(p) = 0$. (Se întâmplă la $p \approx 0.77$, nu la $p = 0.5$. La $p = 0.5, R = 1/1.5 \approx 0.667$).

11. Considerăm că avem un alfabet cu N simboluri. Am discutat la curs despre faptul că entropia este maximizată dacă avem "incertitudine maximă" deci avem un alfabet în care simbolurile au aceeași probabilitate de apariție (adică $p_i = 1/N$ pentru $i = 1, \dots, N$ aceasta se numește distribuția uniformă). Demonstrați matematic această afirmație.

R: Vrem să maximizăm $H(P) = -\sum_{i=1}^N p_i \log_2 p_i$ sub constrângerea $\sum_{i=1}^N p_i = 1$. Folosim Inegalitatea lui Gibbs (sau Multiplicatori Lagrange). Fie $P = \{p_1, \dots, p_N\}$ distribuția noastră și $U = \{1/N, \dots, 1/N\}$ distribuția uniformă. Divergența Kullback-Leibler $D_{KL}(P||U) \geq 0$.

$$D_{KL}(P||U) = \sum_{i=1}^N p_i \log_2 \frac{p_i}{u_i} = \sum p_i \log_2 p_i - \sum p_i \log_2 (1/N) \geq 0$$

$$-H(P) - \log_2(1/N) \sum p_i \geq 0$$

$$-H(P) + \log_2(N) \cdot 1 \geq 0$$

$$\log_2(N) \geq H(P)$$

Astfel, entropia $H(P)$ este întotdeauna mai mică sau egală cu $\log_2(N)$, care este entropia distribuției uniforme. Egalitatea are loc când $P = U$.

12. Avem un alfabet cu N simboluri x_i , fiecare cu probabilitatea de apariție p_i . Considerăm că simbolurile sunt sortate în ordinea descrescătoare a probabilităților de apariție: adică $p_1 \geq p_2 \geq \dots \geq p_N$. Codarea acestor simboluri se realizează în felul următor: simbolul x_i este codat cu șirul binar 00001 (de $i - 1$ ori bitul 0, apoi bitul 1). Sunt relevante valorile p_i în acest caz? Care este lungimea medie a unui mesaj codat în acest fel? Comparați această valoare cu entropia. Care este avantajul principal al acestei codări?

- **Relevanța p_i :** Valorile p_i sunt **irelevante pentru construcția codului** (care depinde doar de indexul i), dar sunt **critice pentru calculul lungimii medii** și pentru a justifica sortarea (simbolul cel mai probabil p_1 primește codul cel mai scurt "1").
- **Lungimea medie (L):** Lungimea codului x_i este $L_i = i$. Lungimea medie este $L = \sum_{i=1}^N p_i \cdot L_i = \sum_{i=1}^N i \cdot p_i$.
- **Comparație:** Din Teorema lui Shannon, știm că $L \geq H(X)$. Deoarece simbolul cel mai probabil (p_1) are lungimea 1, iar cel mai puțin probabil (p_N) are lungimea N , acest cod (Codare Unară) este eficient doar dacă probabilitățile scad foarte repede (ex. $p_i \approx 2^{-i}$), dar în general este ineficient.
- **Avantaj principal: Simplitatea decodării.** Decodificatorul citește biți '0' până când întâlnește un '1'. Numărul de biți '0' citați îi dă direct indexul $i - 1$ al simbolului (sau numărul total de biți citați i dă indexul i).

Seminar 0x02

1. Calculați tabelele de adevăr pentru următoarele expresii logice:

- (a) $X = A + BC$
- (b) $X = A(B + D)$
- (c) $X = (A + B)(A + C)$
- (d) $X = \overline{A}(B + C)D$
- (e) $X = \overline{AB}(C + D)$
- (f) $X = AB(C \oplus D)$
- (g) $X = \overline{ABC}\overline{D}$
- (h) $X = (\overline{AB} + \overline{AB}) + (\overline{CD} + \overline{CD})$
- (i) $X = (\overline{A} \overline{B} + \overline{AB})(\overline{C} \overline{D} + \overline{CD})$
- (j) $X = A(A \oplus B)(B \oplus C)D$

(a) $X = A + BC$

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(b) $X = A(B + D)$

A	B	D	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(c) $X = (A + B)(A + C)$

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(d) $X = \overline{A}(B + C)D$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

(e) $X = \overline{AB}(C + D)$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

(f) $X = AB(C \oplus D)$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

(g) $X = \overline{ABC}\overline{D}$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

(h) $X = (\overline{AB} + \overline{AB}) + (\overline{CD} + \overline{CD})$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

(i) $X = (\overline{A} \overline{B} + \overline{AB})(\overline{C} \overline{D} + \overline{CD})$

A	B	C	D	X
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

(j) $X = A(A \oplus B)(B \oplus C)D$

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

2. Desenați circuitele digitale pentru următoarele expresii logice:

- (a) $X = (A + B)C$
- (b) $X = A + BC + \overline{D}$
- (c) $X = AB + \overline{A} \overline{C}$
- (d) $X = \neg(A + B)(C + D)\overline{D}$
- (e) $X = A \oplus B \oplus CD$
- (f) $X = \overline{AB} \oplus CB$

3. Legile de Morgan sunt: $\overline{A + B} = \overline{A} \overline{B}$ și $\overline{AB} = \overline{A} + \overline{B}$. Cerințe:

- (a) verificați cele două legi ([Explicație detaliată aici](#));

$\overline{A + B} = \overline{A} \cdot \overline{B}$				$\overline{AB} = \overline{A} + \overline{B}$			
A	B	$\overline{A + B}$	$\overline{A} \cdot \overline{B}$	A	B	$\overline{AB}$	$\overline{A} + \overline{B}$
0	0	1	1	0	0	1	1
0	1	0	0	0	1	1	1
1	0	0	0	1	0	1	1
1	1	0	0	1	1	0	0

- (b) desenați circuitele pentru: $\overline{A + B}$, $\overline{A} \overline{B}$, $\overline{AB}$ și $\overline{A} + \overline{B}$;
- (c) folosind și cele două legi, calculați/simplificați expresiile:
- i. $\neg(\overline{A} + \overline{B}) \equiv \textcolor{violet}{AB}$

- ii. $\neg(\overline{A} \overline{B}) \equiv A + B$
- iii. $\overline{A + B + C}$, generalizați la N variabile digitale A_i ; $\overline{A_1 + \dots + A_n} \equiv \overline{A_1} + \overline{A_2} + \dots + \overline{A_n}$
- iv. $\overline{ABC}$, generalizați la N variabile digitale A_i ; $\overline{A_1 \dots A_n} \equiv \overline{A_1} \cdot \overline{A_2} \cdot \dots \cdot \overline{A_n}$
- v. $\neg(A + B) \cdot \overline{A} \cdot \overline{B} \equiv \neg A \cdot \neg B \cdot \neg A \cdot \neg B \equiv \neg A \neg B$
- vi. $\overline{AB(A + B)} \equiv$
- vii. $\overline{(A + B)(A + B)}$
- viii. $\overline{A \cdot B \cdot (AB)}$
- ix. $\overline{C + \overline{CB}}$
- x. $\overline{\overline{AB(A + B)}(\overline{B} + \overline{B})}$
- xi. $\overline{(\overline{AB})(\overline{B} + C)}$
- xii. $\overline{(A + \overline{B})(\overline{B} \cdot C)}$
- xiii. $\overline{(\overline{A} + C)(\overline{AB})}$

4. Simplificați următoarele expresii logice:

- (1) $A + 0 \equiv A$
- (2) $\overline{A} \cdot 0 \equiv 0$
- (3) $A + \overline{A} \equiv 1$
- (4) $A + A \equiv A$
- (5) $A + AB \equiv A(1 + B) \equiv A$
- (6) $A + \overline{A} \cdot B \equiv A + B$
- (7) $A(\overline{A} + B) \equiv AB$
- (8) $AB + \overline{AB} \equiv B(A + \overline{A}) \equiv B$
- (9) $(\overline{A} + \overline{B})(\overline{A} + B) \equiv !A + !A!B + !A \cdot B + !B \cdot B \equiv !A(!B + B) + 0 \equiv !A$
- (10) $A(A + B + C + \dots) \equiv A$
- (11) fie $f(A, B, C) = A + B + C$
 - i.* $f(A, B, AB) \equiv A(1 + B) + B \equiv A + B$
 - i.* $f(A, B, \overline{A} \cdot \overline{B}) \equiv (A + B) + (\overline{A} + \overline{B}) \equiv 1$
 - i.* $f(A, B, \overline{AB}) \equiv (A + !A) + (B + !B) \equiv 1$
- (12) $A + A \cdot \overline{A} \equiv A(1 + \overline{A}) \equiv A$
- (13) $AB + \overline{AB} \equiv A(B + \overline{B}) \equiv A$
- (14) $\overline{A} + B \cdot \overline{A} \equiv !A(1 + B) \equiv !A$
- (15) $(D + \overline{A} + B + \overline{C})B \equiv B$
- (16) $(A + \overline{B})(A + B) \equiv A + A \cdot \overline{B} + A \cdot B + B \cdot \overline{B} \equiv A(1 + B + \overline{B}) + 0 \equiv A$
- (17) $C(C + CD) \equiv C^2(1 + D) \equiv C$
- (18) $A(A + AB) \equiv A^2(1 + B) \equiv A$
- (19) $!A + !A \equiv !(A) \equiv A$
- (20) $A + !A \equiv !1 \equiv 0$
- (21) $D + (D \cdot \overline{C} \cdot B \cdot A) \equiv D$
- (22) $!D \cdot (\overline{DCBA}) \equiv !D(!D + !C + !B + !A) \equiv D$
- (23) $AC + \overline{A} \cdot B + BC \equiv AC + \overline{A} \cdot B$
- (24) $(A + C)(\overline{A} + B)(B + C) \equiv (\overline{A} + B)(B + C)A + (\overline{A} + B)(B + C)C \equiv (B + C) \cdot A \cdot \overline{A} + (B + C)AB + (\overline{A} + B)(B + C)C \equiv 0 + (B + C)AB + (\overline{A} + B)(B + C)C \equiv ABB + ABC + (\overline{A} + B)(B + C)C \equiv AB + ABC + (\overline{A} + B)(BC + CC) \equiv AB + (\overline{A} + B)C \equiv AB + BC + \overline{A}C \equiv AB + \overline{A}C$
- (25) $\overline{A} + \overline{B} + A \cdot B \cdot \overline{C} \equiv \overline{B} + (\overline{A} + B\overline{C}) \equiv \overline{A} + \overline{B} + \overline{C}$
- (26) $(A + C)(A \cdot D + A \cdot \overline{D}) + AC + C \equiv (A + C)A(D + \overline{D}) + C(1 + A) \equiv (A + C) \cdot A \cdot 1 + C \equiv A + C$
- (27) $\overline{A} \cdot (A + B) + (B + AA)(A + \overline{B}) \equiv (\overline{A} \cdot A + \overline{A} \cdot B) + (A + B)(A + \overline{B}) \equiv (0 + \overline{A} \cdot B) + (A \cdot A + B \cdot A + A \cdot \overline{B} + B \cdot \overline{B}) \equiv \overline{A} \cdot B + (A + A(B + \overline{B}) + 0) \equiv A$
- (28) $(A + B)^2 + (A + B)^3 + A + 3\overline{A} + A^3 \equiv (A + B) + (A + B) + (A + 3\overline{A} + A) \equiv (A + B) + (A + \overline{A}) \equiv A + B + 1 \equiv 1$

Resurse relevante pentru algebra booleană din spatele circuitelor logice digitale:

- en.wikipedia.org/wiki/Consensus_theorem
- www.electronics-tutorials.ws/boolean/bool_6.html
- en.wikipedia.org/wiki/Absorption_law

5. Multiplexare; cerințe:

- (a) presupunem că avem un MUX cu două intrări binare și un semnal de un bit de selecție S. Scrieți relația ieșire-intrare pentru acest MUX. Desenați și circuitul digital corespunzător;
- (b) un MUX cu două intrări binare și un semnal de selecție binar (1 bit) poate simula porțile NOT, OR și AND. Desenați MUX-urile (și scrieți tabelul de adevăr) pentru fiecare poartă;

6.

7.

8.

9.

10.

11.

12.

(a)

(b)

(c)

(d)

(e)

(f)

13.

14.

15. Considerăm două variabile binare A și B pe 3 biți fiecare. Scrieți funcții logice care au valoarea "1" doar dacă:

- (a) A conține un singur bit "0";
- (b) A conține un număr impar de biți "0";
- (c) A este un număr mai mic de 4, în baza zecimală;
- (d) $A == B$;
- (e) $A < B$, dacă A și B sunt numere naturale;
- (f) $A < B$, dacă A și B sunt numere întregi scrise în complement față de doi.

16. Arătați că porțile logice NOR și NAND, respectiv, sunt complete din punct de vedere funcțional. Adică, arătați că orice funcție logică poate fi implementată folosind doar porți NOR sau doar porți NAND.

- Poartă NOR:** Poarta NOR este o poartă logică care produce o ieșire "1" doar atunci când toate intrările sunt "0". Funcția sa logică este dată de: $Y = \overline{A + B}$ pentru două intrări A și B.
- Poartă NAND:** Poarta NAND este o poartă logică care produce o ieșire "0" doar atunci când toate intrările sunt "1". Funcția sa logică este dată de: $Y = \overline{AB}$ pentru două intrări A și B.

17. Avem un automat care vinde snack-uri. Acesta acceptă monede de 10 bani, 50 bani și 1 leu. Când o monedă este introdusă în mașină, un semnal digital este activat în funcție de monedă (A, B și respectiv C) și un semnal digital central (G) este activat (o monedă a fost introdusă). Există un buton de resetare (RST) care resetează suma introdusă la zero. Vrem un circuit care activează o ieșire digitală (X) atunci când suma introdusă depășește 60 de bani.

Seminar 0x03

1.

- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

Seminar 0x04

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

Seminar 0x05

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.